

main_search.py

Guía de estudio para la disertación doctoral

Búsqueda evolutiva multi-objetivo (NSGA-II) de arquitecturas neuronales cuantizadas para detección de anomalías en MVTec-AD, orientada a plataformas embebidas (Jetson Orin Nano)

Versión del código: revisión y correcciones del 11 de junio de 2026 · Documento generado a partir del análisis línea a línea de main_search.py y de sus módulos src/nas/*

Contenido	
1.	Qué es y qué papel cumple en la tesis
2.	El problema formal de optimización multi-objetivo
3.	Anatomía del script: clases, funciones y flujo de ejecución
4.	El genoma de 48 genes y el espacio de búsqueda
5.	La función de fitness: evaluación proxy de cada candidato
6.	El motor NSGA-II paso a paso
7.	Artefactos de salida y encadenamiento con el pipeline
8.	Correcciones aplicadas para la re-ejecución
9.	Cómo re-ejecutar la búsqueda (receta completa)
10.	Teoría que debes estudiar (apuntes con referencias)
11.	Preguntas probables del jurado y cómo abordarlas

1. Qué es y qué papel cumple en la tesis

main_search.py es el corazón metodológico del proyecto: implementa la etapa de **búsqueda conjunta de arquitectura y cuantización** mediante el algoritmo evolutivo multi-objetivo NSGA-II. Su salida (los candidatos Pareto-óptimos en best_candidates.json) alimenta el reentrenamiento (main_retrain.py), el despliegue embebido (main_deploy.py) y la validación en seguimiento visual (main_tracking.py).

En términos de tus objetivos doctorales, este script materializa el **objetivo específico 2**: "proponer y evaluar un esquema de optimización multi-objetivo para la determinación conjunta de arquitectura, representación numérica y configuración de cuantización". La palabra clave es **conjunta**: en lugar del enfoque secuencial tradicional (primero diseñar/buscar la red en precisión completa y después cuantizarla), aquí cada individuo de la población evolutiva codifica al mismo tiempo la topología de la red Y su esquema de cuantización por capa, y ambos se evalúan juntos contra los cuatro objetivos. La hipótesis de la tesis es que este co-diseño encuentra mejores compromisos que la secuencia diseño-luego-cuantización, porque la sensibilidad a la cuantización depende de la arquitectura (una red que es óptima en FP32 puede degradarse mucho en INT4/INT8, mientras otra ligeramente peor en FP32 puede ser casi insensible).

El dominio de aplicación es la **detección de anomalías industriales** sobre MVTec-AD: el modelo aprende solo con imágenes normales (entrenamiento no supervisado/one-class) y en inferencia produce una puntuación de anomalía por imagen; el AUROC mide cuán bien esa puntuación separa imágenes buenas de defectuosas.

2. El problema formal de optimización multi-objetivo

El script resuelve el siguiente problema: encontrar el conjunto de configuraciones x (genomas enteros de 48 genes) que minimizan simultáneamente el vector de objetivos

```
F(x) = ( f1, f2, f3, f4 )
f1 = -AUROC(x) (se maximiza AUROC, se niega para minimizar)
f2 = latencia_ms(x) (inferencia, en el dispositivo de búsqueda)
f3 = RAM_pico_MB(x) (memoria atribuible al modelo)
f4 = energía_mJ(x) (energía por inferencia, NVML en PC / tegrastats en Jetson)
```

Los cuatro objetivos están en **conflicto**: subir AUROC suele exigir redes más anchas y profundas (más latencia, RAM y energía); bajar los bits de cuantización reduce memoria y energía pero puede degradar AUROC. No existe en general una solución que sea óptima en todo a la vez, así que el resultado correcto no es un único modelo sino un **frente de Pareto**: el conjunto de soluciones no dominadas.

Dominancia de Pareto (definición que debes saber de memoria): una solución a domina a b si a no es peor que b en ningún objetivo y es estrictamente mejor en al menos uno. El frente de Pareto es el conjunto de soluciones que nadie domina. En el código, esta definición está implementada literalmente en nsga2_engine.dominates() y es la base del ordenamiento no dominado.

Por qué no escalarizar: la alternativa simple (combinar los 4 objetivos en una suma ponderada) obliga a elegir pesos a priori, encuentra una única solución por corrida y no puede alcanzar regiones no convexas del frente. NSGA-II aproxima TODO el frente en una sola corrida, y la decisión final (qué punto del frente desplegar) se pospone a la etapa de despliegue con métricas reales del dispositivo. Este argumento es central para la disertación.

3. Anatomía del script: clases, funciones y flujo

3.1 Mapa de componentes

Componente (líneas aprox.)	Responsabilidad
SearchConfig (dataclass)	Toda la configuración de la corrida: rutas, semilla, tamaño de población (32), generaciones (30), probabilidades de cruce/mutación, hv_reference, penalty_objectives, top_k, resume_from, keep_old, allow_noop_energy. Se carga desde config/search.yaml (plano, 1:1 con los campos) y los flags de la CLI tienen prioridad.
HistoryWriter	Escritor CSV con apertura perezosa en modo append: solo abre el archivo en el primer write y nunca trunca (la versión anterior abría con 'w' en el constructor y por eso history.csv quedó en 0 bytes). Se usa para history.csv y para el nuevo evaluations.csv.
_clean_previous_outputs()	Borra los artefactos de una corrida anterior (population_gen_*.csv, history.csv, evaluations.csv, pareto_front.csv, best_candidates.json, checkpoints, resúmenes) antes de un run fresco. Se omite si hay resume_from o --keep-old. Los .log no se tocan (el logger los tiene abiertos).
SearchPipeline	Orquestador. _build_components() construye SearchSpace -> GenomeEncoder -> FitnessEvaluator (con el callback de registro por evaluación); _probe_energy_backend() verifica el medidor de energía ANTES de gastar horas de cómputo; _on_generation_end() persiste población, historia y checkpoint por generación; _finalize() calcula el frente final y escribe pareto_front.csv y best_candidates.json; run() encadena todo.
NSGA2Engine (src/nas/nsga2_engine.py)	El algoritmo evolutivo en sí: inicialización, bucle generacional (mu+lambda), sort no dominado, crowding distance, torneo, cruce, mutación, selección ambiental, hipervolumen Monte Carlo, checkpoint/restauración.
GenomeEncoder (src/nas/encoding.py)	Traducción genotipo-fenotipo: vector entero de 48 genes <-> dict {arch_spec, quant_spec} que consumen model_factory y qat_wrapper. También fingerprint() (hash hex del genoma) para deduplicación y trazabilidad.
FitnessEvaluator (src/nas/fitness.py)	Evalúa UN candidato: construye el modelo, aplica fake-quant, entrenamiento corto, AUROC, perfilado de hardware. Devuelve {auroc, latency_ms, peak_ram_mb, energy_mj} o penaliza con razón explícita.
CLI (_build_argparser / main)	Flags: --config, --results-dir, --category, --seed, --population-size, --n-generations, --device, --resume-from, --top-k, --keep-old, --allow-noop-energy, --quiet.

3.2 Flujo de ejecución (de principio a fin)

#	Paso	Detalle
1	Parseo de CLI y carga de config	Si config/search.yaml existe se carga (esquema plano); si no, defaults internos + WARNING. Los flags CLI sobrescriben campos individuales.
2	Logging dual	Consola + results/search/search.log con timestamps.

#	Paso	Detalle
3	Limpieza de la corrida anterior	Salvo resume_from o --keep-old. Garantiza que resultados viejos y nuevos jamás se mezclen.
4	Sonda del medidor de energía	Mide un Conv2d diminuto (5 iteraciones). Si el backend es 'noop' (sin NVML/tegrastats) ABORTA con RuntimeError: evita repetir la corrida piloto donde energy_mj=9999 constante degeneró la búsqueda a 3 objetivos sin que nadie lo notara.
5	Persistencia de la config resuelta	search_config.json (reproducibilidad).
6	Construcción de componentes	SearchSpace (espacio por defecto o restringido vía YAML search_space:{}) -> GenomeEncoder -> FitnessEvaluator con extra_callbacks=[_log_evaluation]. mutation_prob = 1/48 si es null.
7	Creación del motor	NSGA2Config recibe población, prob. de cruce, tasa de mutación, semilla y AHORA también hv_reference y penalty_objectives desde la config (antes usaba defaults internos desalineados).
8	Inicialización / reanudación	Sin checkpoint: población aleatoria de 32 genomas y evaluación completa (generación 0). Con --resume-from: restaura población, objetivos, generación e historia desde latest_checkpoint.{npz,json}.
9	Bucle generacional	engine.run(n_generations restantes, on_generation_end). Tras cada generación el callback escribe population_gen_NNN.csv, agrega filas a history.csv, loggea métricas y guarda latest_checkpoint.
10	Finalización	Sort no dominado sobre la población final -> pareto_front.csv (con crowding distance) y best_candidates.json (top-5 del frente ordenado por AUROC y luego latencia) -> entrada de main_retrain.py. Además final_population.npz y search_summary.json.

4. El genoma de 48 genes y el espacio de búsqueda

Cada candidato es un vector entero de longitud fija 48. Cada gen es un ÍNDICE dentro de una lista de opciones discretas (no un valor crudo), de modo que todo el vector vive en un hipercubo entero fijo: ideal para operadores de mutación enteros y para serializar. El layout (fuente: `src/nas/search_space.py`, sección autoritativa):

Genes	Campo	Opciones (lista indexada)
0	arch_family	autoencoder, unet, feature_recon, student_teacher, patch_cnn
1	input_size	128, 192, 224, 256
2	n_stages (profundidad)	entero en [2, 5]
3-7	channels (por etapa, 5 slots)	16, 24, 32, 48, 64, 96, 128, 192, 256
8-12	kernel (por etapa)	3, 5, 7
13-17	block_type (por etapa)	conv (estándar), ds (depthwise-separable), ir (inverted residual)
18-22	bottleneck_ratio (por etapa)	0.5, 1.0, 2.0, 4.0, 6.0
23-27	skip_connections (por etapa)	binario {0,1}
28-32	attention (por etapa)	none, se, eca, spatial, cbam
33	global_weight_bits	4, 6, 8
34	global_act_bits	4, 6, 8
35	symmetric	binario {0,1}
36	per_channel	binario {0,1}
37	mixed_precision	binario {0,1}
38-42	layer_weight_bits (por etapa)	4, 6, 8 (solo activos si mixed_precision=1)
43-47	layer_act_bits (por etapa)	4, 6, 8 (solo activos si mixed_precision=1)

Dos sutilezas que el jurado puede preguntar: (1) el genoma siempre mide 48 aunque `n_stages` sea menor que 5 — los slots inactivos existen y mutan, pero el decodificador los ignora (representación redundante con genes 'silenciosos', análoga al ADN no codificante; es inofensiva y simplifica los operadores); (2) los genes de cuantización por capa solo tienen efecto cuando `mixed_precision=1`; si no, mandan los bits globales. El tamaño del espacio es del orden de $5 \cdot 4 \cdot 4 \cdot (9 \cdot 3 \cdot 3 \cdot 5 \cdot 2 \cdot 5)^5 \cdot (3 \cdot 3 \cdot 2 \cdot 2 \cdot 2) \cdot (3 \cdot 3)^5$, es decir $>10^{20}$ combinaciones: imposible de enumerar, de ahí la búsqueda evolutiva.

La traducción genotipo->fenotipo la hace `GenomeEncoder.gene_to_dict()`, que produce el dict `{arch_spec, quant_spec}` consumido por `model_factory.build_model()` (construye la red PyTorch según la familia: autoencoder reconstruye la imagen, student_teacher compara embeddings profesor-alumno, patch_cnn puntúa parches, etc.) y por `qat_wrapper.wrap_for_qat()` (inserta nodos de cuantización simulada).

5. La función de fitness: evaluación proxy de cada candidato

Evaluar de verdad un candidato costaría horas (entrenamiento completo). La búsqueda usa una **evaluación proxy de baja fidelidad** cuyo propósito NO es estimar el AUROC final sino RANQUEAR candidatos de forma consistente. `FitnessEvaluator.evaluate()` ejecuta:

Etapa	Qué hace	Penalización si falla
1. Construcción	build_model(arch_spec) crea la red PyTorch.	BUILD_FAILED
2. Restricciones duras	Máximo 20 M de parámetros (max_params_m).	CONSTRAINT_VIOLATED
3. QAT fake-quant	wrap_for_qat(quant_spec) inserta cuantización simulada; calibración de observadores con 5 batches.	QAT_FAILED
4. Entrenamiento corto	5 épocas, máximo 500 muestras normales, batch 8, lr 1e-3, weight decay 1e-4, clip de gradiente 1.0; timeout 600 s; pérdida NaN o >1e6 = divergencia.	TRAINING_DIVERGED / TIMEOUT / OOM
5. AUROC	Puntuación de anomalía sobre el split de test de la categoría; AUROC imagen-nivel.	EVAL_FAILED
6. Perfilado HW	Latencia (30 iteraciones, 10 de warmup), RAM atribuible al modelo (pico del asignador CUDA o delta de RSS del host — corrección de esta revisión), energía por inferencia (NVML en PC, tegrastats en Jetson); timeout 120 s.	PROFILING_FAILED
7. Normalización	Min-max contra referencias best/worst (solo informativa para logs).	-
8. Retorno	{auroc, latency_ms, peak_ram_mb, energy_mj} al motor; el callback escribe la fila completa (con razón de penalti y tiempos) en evaluations.csv.	-

Los valores de penalización (AUROC=0, latencia=9999 ms, RAM=65536 MB, energía=9999 mJ) son 'peores que cualquier valor real', de modo que los individuos fallidos quedan dominados por cualquier individuo viable y la evolución los elimina. Tras la corrección de hoy, el motor usa EXACTAMENTE el mismo vector de penalti, y esos puntos caen fuera del punto de referencia del hipervolumen, así que un fallo jamás infla el indicador.

Justificación del proxy (pregunta segura del jurado): el supuesto es que el ranking con 5 épocas correlaciona con el ranking con entrenamiento completo (correlación de Kendall tau en la literatura de NAS). La validación empírica está en tu propio pipeline: los 5 candidatos del frente reentrenados 200 épocas en main_retrain alcanzaron AUROC 0.90-0.914 partiendo de proxies de 0.876-0.894, preservando el orden aproximado.

6. El motor NSGA-II paso a paso

NSGA-II (Non-dominated Sorting Genetic Algorithm II, Deb et al. 2002) es el estándar de facto en optimización evolutiva multi-objetivo. La implementación del proyecto (src/nas/nsga2_engine.py) sigue el algoritmo canónico con un esquema ($\mu + \lambda$):

Fase	Implementación en el código
Inicialización	initialize(): 32 genomas aleatorios muestreados del espacio (sample_population) y evaluados. Es la generación 0 y la parte más cara junto con cada generación posterior (32 evaluaciones proxy).
1. Ordenamiento no dominado rápido	fast_non_dominated_sort(): particiona la población en frentes F1, F2, ... donde F1 = no dominados, F2 = no dominados al quitar F1, etc. Complejidad $O(M \cdot N^2)$ con $M=4$ objetivos, $N=64$ (población + descendencia).
2. Crowding distance	crowding_distance(): dentro de cada frente, mide cuán aislado está cada punto (suma de las distancias normalizadas a los vecinos por objetivo; infinito en los extremos). Es el mecanismo de preservación de DIVERSIDAD: ante igual rango, se prefiere el punto más aislado para extender el frente.
3. Selección por torneo binario	_tournament_select(): se sortean 2 individuos; gana el de menor rango de Pareto y, a igualdad de rango, el de mayor crowding distance (comparación 'crowded' del paper original).
4. Cruce	_create_offspring(): cruce de un punto (o uniforme, configurable) con probabilidad 0.9 sobre los vectores enteros de 48 genes. ATENCIÓN: los parámetros eta_c/eta_m del YAML corresponden a SBX y mutación polinomial (operadores para variables REALES) y NO se usan; el genoma es entero. Tenlo claro en la defensa: es un punto fácil de cuestionar si el manuscrito dice 'SBX'.
5. Mutación	Reinicio aleatorio por gen con probabilidad 1/48 (en promedio, un gen mutado por hijo): el gen toma un índice aleatorio válido de su lista de opciones. Variante de vecindad (+1 paso) disponible pero desactivada.
6. Selección ambiental elitista	_select_next_generation(): se apilan padres + hijos ($2N=64$), se ordenan por frentes y se llenan las 32 plazas frente a frente; el frente que no cabe completo se trunca por crowding distance descendente. El elitismo garantiza que un buen punto del frente nunca se pierde por azar.
7. Indicador de progreso	Hipervolumen del frente F1 estimado por Monte Carlo (50 000 muestras uniformes dentro de la caja definida por el punto de referencia; HV = fracción de muestras dominadas x volumen de la caja). Es la métrica de convergencia que se reporta por generación junto con mejor AUROC, latencia mínima, tamaño del frente y número de fallos.

El punto de referencia del hipervolumen — y la lección del piloto: el HV solo cuenta el volumen dominado ENTRE el frente y el punto de referencia r . Si una solución real queda 'peor' que r en algún objetivo, su contribución es cero. En la corrida piloto, r tenía energía=1000 mJ pero TODAS las soluciones registraban energía=9999 (penalti por medidor ausente): el HV quedó degenerado y, con un objetivo constante y otro casi constante (RAM de proceso), 32 de 32 individuos terminaron 'no dominados' — un frente inflado artificialmente. Por eso ahora (a) la sonda de energía aborta si no hay medidor, (b) r es configurable (hv_reference) con default holgado para PC [0, 1000 ms, 8192 MB, 5000 mJ], y (c) los penaltis caen fuera de r a propósito. Calibra r tras la primera generación con evaluations.csv: debe ser ligeramente peor que el peor valor factible observado en cada objetivo.

7. Artefactos de salida y encadenamiento

Archivo (results/search/)	Contenido	Quién lo consume
population_gen_NNN.csv	Los 32 individuos de la generación NNN: objetivos + candidato JSON.	Análisis de convergencia; reconstrucción de historia; figuras del paper.
history.csv	Una fila por individuo por generación (vista del motor).	main_report (curvas de convergencia).
evaluations.csv (nuevo)	Una fila por LLAMADA al evaluador, con timestamp, los 4 objetivos crudos y normalizados, razón de penalti, épocas entrenadas, duración. Es la traza primaria de la búsqueda.	Calibración de hv_reference; análisis de fallos; estimación de costo computacional; convergencia para el paper.
latest_checkpoint.{npz,json}	Población + objetivos + generación + historia del motor; se sobrescribe cada generación.	Reanudación con --resume-from tras un corte.
pareto_front.csv	El frente final con crowding distance y los genomas decodificados.	main_report; selección manual de candidatos.
best_candidates.json	Top-5 del frente (orden lexicográfico: AUROC desc., latencia asc.).	main_retrain.py (entrada directa de la siguiente etapa).
final_population.npz	Genomas + objetivos finales en binario numpy.	Warm-start de búsquedas futuras.
search_config.json / search_summary.json	Config resuelta y resumen (duración, tamaño del frente, etc.).	Reproducibilidad; tabla de setup del paper.

8. Correcciones aplicadas para la re-ejecución (11-jun-2026)

#	Corrección	Por qué importa
1	Sonda de energía al arranque: aborta si el backend es 'noop' (--allow-noop-energy para anular). Requiere 'pip install nvidia-ml-py' en el PC.	La piloto corrió 30 generaciones con energía=9999 constante: optimización de 3 objetivos disfrazada de 4. Irrepetible para un paper Q1.
2	RAM atribuible al modelo (pico del asignador CUDA / delta RSS) en fitness.py.	El RSS absoluto (~1.8 GB) era casi constante entre candidatos: objetivo no informativo.
3	hv_reference y penalty_objectives expuestos en SearchConfig/YAML y pasados al motor; penaltis del motor alineados con los del evaluador (9999/65536/9999).	Con el default interno (energía ref=1000 mJ) los valores reales del PC (hasta ~2000 mJ/inf) habrían quedado fuera de la referencia: HV degenerado otra vez, incluso con el medidor funcionando.
4	evaluations.csv: traza por evaluación vía extra_callbacks del FitnessEvaluator.	Sin ella no hay evidencia de convergencia ni análisis de fallos (history.csv del piloto quedó vacío). Es el insumo del análisis exigido por el objetivo específico 2.
5	HistoryWriter con apertura perezosa en append.	El truncado a 0 bytes ocurría al reanudar con 0 generaciones restantes.
6	Limpieza automática de artefactos viejos al iniciar run fresco (--keep-old para conservar); la corrida piloto fue archivada en results/search_pilot_2026-05/.	Evita mezclar métricas de corridas distintas en los mismos CSV.
7	config/search.yaml plano que SÍ se carga (los main apuntaban a configs/ inexistente).	El piloto corrió con defaults internos; el YAML era decorativo.

9. Cómo re-ejecutar la búsqueda (receta completa)

Paso 0 — prerequisite único pendiente:

```
pip install nvidia-ml-py
```

Paso 1 — humo (2-10 min): verifica energía real, limpieza y CSVs con una mini-corrida. Revisa en el log la línea "Energy meter backend OK: source=nvml" y que results/search/evaluations.csv tenga filas con energy_mj distinto de 9999:

```
python main_search.py --population-size 8 --n-generations 1
```

Paso 2 — calibración del hipervolumen: con evaluations.csv de la generación 1, ajusta hv_reference en config/search.yaml a valores ~20-50% peores que el peor valor factible observado por objetivo (manteniéndolos por debajo de los penaltis).

Paso 3 — corrida completa (32 x 30; estimación: 990 evaluaciones a 30-90 s cada una en la RTX 3050, es decir entre 8 y 25 horas; evaluations.csv te da el costo real tras la primera generación):

```
python main_search.py --config config/search.yaml
```

Paso 4 — si se corta (apagón, Ctrl+C): reanuda desde el último checkpoint; la limpieza se omite automáticamente al reanudar:

```
python main_search.py --resume-from results/search/latest_checkpoint.npz
```

Paso 5 — verificación de salida: `pareto_front.csv` con un frente de tamaño razonable (típicamente 5-15 puntos con 4 objetivos vivos; si vuelve a salir 32/32 sospecha de un objetivo degenerado), `best_candidates.json` con 5 candidatos, y las cuatro columnas de objetivos con varianza real entre individuos. Después: `main_retrain.py`.

10. Teoría que debes estudiar (apuntes con referencias)

Lista priorizada. Para cada tema: qué dominar para la disertación y la referencia canónica por la que empezar.

Tema	Qué debes dominar	Referencia de partida
T1. Optimización multi-objetivo	Dominancia de Pareto, frente/conjunto de Pareto, puntos ideal y nadir, por qué la escalarización ponderada no cubre frentes no convexos.	K. Deb, 'Multi-Objective Optimization using Evolutionary Algorithms', Wiley, 2001 (caps. 1-2).
T2. NSGA-II	Las 4 piezas: fast non-dominated sort (y su complejidad $O(M \cdot N^2)$), crowding distance, torneo 'crowded', elitismo ($\mu + \lambda$). Saber recorrer una generación a mano con un ejemplo de 6 puntos.	Deb, Pratap, Agarwal, Meyarivan, 'A Fast and Elitist Multiobjective GA: NSGA-II', IEEE Trans. Evolutionary Computation, 2002. (El paper más citado del área; léelo completo.)
T3. Indicadores de calidad	Hipervolumen: definición, sensibilidad al punto de referencia, estricta monotonía respecto a la dominancia, estimación Monte Carlo (lo que hace tu código). Complementos: spacing, IGD.	Zitzler & Thiele, 'Multiobjective EAs: A Comparative Case Study and the Strength Pareto Approach', IEEE TEC, 1999.
T4. Neural Architecture Search	Taxonomía (espacio de búsqueda / estrategia / estimación de desempeño), NAS evolutivo, NAS hardware-aware multi-objetivo (latencia como objetivo, no proxy).	Elsken, Metzen, Hutter, 'Neural Architecture Search: A Survey', JMLR 2019; Real et al., 'Regularized Evolution for Image Classifier Architecture Search', AAAI 2019; Tan et al., 'MnasNet', CVPR 2019; Cai et al., 'Once-for-All', ICLR 2020.
T5. Evaluación de baja fidelidad (proxy)	Por qué 5 épocas bastan para RANQUEAR: correlación de rankings (Kendall tau), riesgos del proxy (bias hacia redes que convergen rápido) y cómo mitigarlo (validación con reentrenamiento completo, que es exactamente tu main_retrain).	White et al., 'How Powerful are Performance Predictors in NAS?', NeurIPS 2021; Falkner et al., 'BOHB', ICML 2018 (multi-fidelidad).
T6. Cuantización de redes	Cuantización uniforme afín/simétrica, por-tensor vs por-canal, PTQ vs QAT, fake-quantization y el straight-through estimator (STE), precisión mixta por capa, sensibilidad por capa.	Jacob et al., 'Quantization and Training of NNs for Efficient Integer-Arithmetic-Only Inference', CVPR 2018; Gholami et al., 'A Survey of Quantization Methods', 2021; Wang et al., 'HAQ: Hardware-Aware Automated Quantization', CVPR 2019; Bengio et al., 2013 (STE).
T7. Detección de anomalías industrial	El protocolo MVTec-AD (entrenar solo con normales), paradigmas: reconstrucción (autoencoder), student-teacher, embeddings/memoria (PatchCore), flujos; AUROC imagen-nivel y píxel-nivel y su lectura probabilística ($P(\text{score_anómalo} > \text{score_normal})$).	Bergmann et al., 'MVTec AD - A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection', CVPR 2019; Bergmann et al., 'Uninformed Students', CVPR 2020; Roth et al., 'Towards Total Recall (PatchCore)', CVPR 2022.

Tema	Qué debes dominar	Referencia de partida
T8. Medición en hardware embebido	Cómo se mide energía (NVML en GPU de escritorio; rieles INA3221/tegrastats en Jetson), por qué fijar relojes (nvpmodel/jetson_clocks), warmup, percentiles de latencia, y por qué los motores TensorRT no son portables.	Documentación NVML y tegrastats de NVIDIA; metodología de benchmarking reproducible (p. ej. MLPerf Inference, Reddi et al., ISCA 2020).

11. Preguntas probables del jurado y cómo abordarlas

Pregunta	Línea de respuesta
¿Por qué NSGA-II y no NSGA-III o MOEA/D, si tienes 4 objetivos?	4 objetivos es la frontera donde NSGA-II sigue siendo competitivo con poblaciones pequeñas (32); NSGA-III aporta con muchos objetivos ($\geq 4-10$) y poblaciones grandes guiadas por puntos de referencia. Argumenta simplicidad, elitismo probado y comparabilidad con la literatura NAS; reconoce NSGA-III como análisis de sensibilidad futuro (tu doc de plan ya lo contempla).
¿Qué fidelidad tiene el proxy de 5 épocas?	Su función es ranquear, no predecir AUROC absoluto. Evidencia interna: el orden del frente se preservó razonablemente tras 200 épocas (0.876-0.894 proxy $\rightarrow$ 0.900-0.914 final). Cita la literatura de performance predictors (T5) y muestra la correlación con tus propios datos.
¿Por qué cruzas con un punto y no con SBX si tu YAML menciona eta_c?	El genoma es entero-categorico (índices), donde SBX (diseñado para reales) no aplica de forma natural; se usa cruce de un punto + mutación por reinicio, operadores estándar para representaciones discretas. eta_c/eta_m quedaron como vestigio de configuración y están marcados como no usados.
¿La cuantización que optimizas es la que despliegas?	En búsqueda es fake-quant (fidelidad de simulación, bits mixtos 4/6/8); en despliegue TensorRT ejecuta FP16/INT8. La brecha se mide explícitamente (AUROC simulado vs on-device, umbral 0.02 en main_deploy) y se reporta como análisis de sensibilidad al error de cuantización.
¿Por qué energía como objetivo y no como restricción?	Como objetivo revela el trade-off completo (frente AUROC-energía) y deja la decisión al desplegador según el presupuesto del robot; una restricción dura fijaría a priori un umbral arbitrario y ocultaría soluciones interesantes cerca del límite.
¿Cómo garantizas que un fallo de evaluación no contamina el resultado?	Triple mecanismo: penalti dominado-por-todos (9999/65536), exclusión del hipervolumen (penalti fuera del punto de referencia) y registro de la razón del fallo en evaluations.csv para auditoría.
¿Qué pasó en la corrida piloto y qué aprendiste?	Respuesta honesta y fuerte metodológicamente: el medidor de energía cayó en silencio a un backend nulo, el objetivo quedó constante y el frente se infló (32/32 no dominados). Se detectó por auditoría de los CSV, se corrigió con verificación activa del backend (fail-fast), alineación de penaltis y trazabilidad por evaluación. Es un ejemplo de por qué el marco reproducible del objetivo específico 1 importa.

Documento generado automáticamente a partir del análisis del código fuente (main_search.py, src/nas/{search_space, encoding, fitness, nsga2_engine, pareto}.py) y de los artefactos de la corrida piloto archivada en results/search_pilot_2026-05/. Las referencias bibliográficas son las canónicas de cada tema; verifica año/venue al citarlas

en el manuscrito.